SharpeArena: The Point-in-Time (PIT) RL Environment for Trading Agents

Tiberiu Toca
General Liquidity
tibi.toca@gmail.com

Abstract

A benchmark scores trajectories; something has to produce them, and the producer is where trading evaluations break. If an agent can read one future bar, or if the same run scores differently on two machines, or if a submitted trajectory cannot be checked against the environment that allegedly produced it, no downstream statistic repairs the damage. We describe SharpeArena, a point-in-time (PIT) RL environment for trading agents built so that these failures are impossible by construction rather than forbidden by rule. The data layer exposes no API that reads a future bar, so an agent cannot peek; a deny-list guard at the harness layer catches tools that try. The determinism-critical core is one Rust engine whose scenario generator uses only multiply, add, divide and max, avoiding transcendental calls that differ across libm implementations, and committed FNV-1a golden hashes pin generated scenarios and order-book tapes byte-for-byte across the native, WebAssembly and Python surfaces. A trajectory records only the agent's decisions; replaying them against the frozen scenario recomputes the result byte-identically, so a doctored trajectory is detectable by anyone. Agents speak a governed wire contract, a JSON Observation in and a JSON Decision out, versioned additively with published schemas, conformance fixtures and a deprecation policy. Scenarios are procedural in the Procgen style with provably disjoint train, gap and held-out seed bands, so overfitting is measurable rather than assumed away. Ranking is delegated to the SharpeBench kernel; the suite also ships a closed-form Avellaneda-Stoikov optimal baseline, mandate-conditioned objectives, and a probe layer for adverse selection, manipulation payoff boundaries and population ecology. Eight committed experiments produce every number in the paper. The corrected scoring kernel rewrites the baseline board: where a unit bug had scored every baseline 0.0000 deflated Sharpe on every tier, drift now saturates the deflated Sharpe at 1.0 on the calm tier, yet no baseline is rank-eligible because none passes the per-run gate on every seed, so the eligibility conjunction, not a mis-united floor, is what gates. Fixed-spread market making pays a U-shaped regret against the provable optimum, from 84.6 at the tightest quote to 0.037 at the closed-form spread. Cross-regime transfer costs the calm-trained reference nearly its entire deflated Sharpe (a 0.97 transfer gap, calm to extreme), the manipulation probe finds pumping unprofitable at every tested impact coefficient and size, and a regime-shock schedule replaces the calm-regime ecology winner with the unlevered long book.

Preprint.

1 Introduction

An eval is useless without an environment. The companion SharpeBench paper [Toca, 2026] argues that ranking trading agents on raw return over short windows measures luck, and supplies a deterministic scoring kernel that deflates for search breadth [Bailey and de Prado, 2014], gates on per-run reliability [Yao et al., 2024], and zeroes entries that bypass risk controls. But a scorer consumes trajectories, and every one of its guarantees is conditional on the trajectory being honest. Three failures upstream of scoring void any score. An agent that observes one future bar has an edge no deflation can remove; look-ahead bugs are the dominant failure mode of backtests, and they are usually invisible in the output. An environment that is not bit-reproducible cannot support a leaderboard, because the same submission scores differently on different machines and no one can check anyone else's number. And a trajectory that cannot be recomputed from the environment that allegedly produced it is a self-reported claim, not evidence.

SharpeArena is the producer built against those three failures, and its design method is uniform: make each failure structurally impossible, then add a detection layer behind the structural guarantee for defense in depth. Look-ahead is prevented by the shape of the data layer, which has no API that returns a future bar, and additionally policed by a deny-list guard at the harness boundary. Determinism is achieved by restricting the scenario generator's arithmetic to operations that are exact and identical on every platform, and additionally pinned by committed golden hashes checked in continuous integration on the native and WebAssembly builds. Trajectory honesty is achieved by recording only decisions and recomputing everything else, and additionally exercised by a test that doctors a trajectory and asserts the replay diverges.

The strategic bet is interface ownership, the move that made Gym the substrate of a decade of reinforcement-learning research [Brockman et al., 2016, Towers et al., 2024]. An agent is a program in any language that reads a JSON `Observation` and writes a JSON `Decision`. The contract is versioned, additive-only, schema-published and governed by a written deprecation policy, so a vendor can build against it once. Conform to the contract and you are scorable everywhere the SharpeBench kernel runs.

Contributions. (1) A leak-free-by-construction, cross-runtime byte-deterministic, replay-verifiable point-in-time (PIT) RL environment for trading agents, with the enforcing code identified for each property. (2) A governed agent wire contract at version 1.0 with JSON Schemas, conformance fixtures and an additive-only evolution policy. (3) A procedural scenario generator with provably disjoint train, gap and held-out seed bands in the Procgen tradition [Cobbe et al., 2020], making generalization a measured quantity. (4) A task suite that includes a market-making environment shipping its closed-form Avellaneda-Stoikov optimum [Avellaneda and Stoikov, 2008] as a baseline, so agents are scored on regret against a provable optimum rather than against each other. (5) A diagnostic layer that probes the simulator itself: stylized-facts realism certification [Cont, 2001], manipulation payoff boundaries, adverse-selection markouts, and population ecology under regime shocks. (6) Eight executed experiments (Section 5) in which every number is produced by a committed script listed in Section A.

What the evidence shows. The strongest finding corrects the benchmark's own floor. Under the pre-0.5.0 scoring kernel every baseline scored a deflated Sharpe of 0.0000 on every tier; that apparent strictness was a unit bug, an annualized deflation prior applied per period, setting the bar near an annualized Sharpe of 18. Under the corrected kernel, drift saturates the deflated Sharpe at 1.0000 on the Calm tier, and still nothing is rank-eligible, because no baseline passes the per-run pass^k gate on every seed (best rate 0.75); on Hard and Extreme the deflated Sharpe collapses to 0.11 and 0.02 (Section 5.1). The eligibility conjunction, deflation and per-run reliability together, is what gates, and each leg catches what the other misses. Second, on the market-making task the environment's regret metric behaves as a provable optimum implies: the closed-form policy scores exactly 0 and fixed-spread quoting pays a U-shaped regret, 84.6 at a 0.05 half-spread, a minimum of 0.037 at 0.5, rising again to 58.8 at 4.0 (Section 5.2). The remaining findings calibrate the generalization

instrument (a 0.97 calm-to-extreme transfer gap against a near-zero within-tier gap, Section 5.3), grade the generator's realism honestly including its failures (Section 5.4), certify that manipulation never pays at any tested impact coefficient or size (Section 5.5), price adverse selection at 0.07 to 0.44 per filled unit by horizon (Section 5.6), map how failure modes shift from structural mandate breaches toward drawdown breaches and stop-outs as stress rises (Section 5.7), and show a regime-shock schedule replacing the calm-regime ecology winner with the unlevered long book (Section 5.8).

2 Design principles

Every decision in the environment traces to one of seven principles, stated here so the rest of the paper reads as their consequences.

Leak-free by construction. The environment owns the time cursor, and the data layer exposes no operation that returns a bar after it. An agent cannot peek because there is nothing to call, not because a rule forbids it. Detection layers exist, but they sit behind the structural guarantee, never in place of it.

Deterministic across runtimes. A published number must be byte-identical on any platform and any surface. The determinism-critical core is one Rust engine; its scenario transform uses only multiply, add, divide and max, because `ln` and `exp` differ across libm implementations. Golden hashes committed in the source pin the outputs, and the WebAssembly build asserts equality with the native engine rather than assuming it.

Recompute, do not trust. A trajectory is evidence only if a third party can recompute it. Runs record the agent's decisions and nothing derived; replay against the frozen scenario reproduces returns byte-for-byte, and a tampered trajectory reproduces something else.

The contract is the product. The agent interface is a tiny JSON surface that evolves additively under a written governance policy, with schemas and conformance fixtures as the authoritative artifacts. Whoever defines the interface owns the ecosystem; an interface that moves under its implementers owns nothing.

Generalization is measured, not presumed. Scenarios are procedural functions of an integer seed, in the Procgen model [Cobbe et al., 2020], and the protocol fixes disjoint train, gap and held-out seed bands. Overfitting becomes one number, the train-minus-test deflated Sharpe, instead of a suspicion.

The environment produces; the kernel judges. SharpeArena computes no rank. Scoring is delegated to the SharpeBench kernel [Toca, 2026], so the deflation, reliability and process gates are specified once, in one place, and the environment cannot quietly disagree with the scorer about what a good run is.

Distrust the simulator too. A synthetic market that pays manipulation without bound, or that lets makers profit from informed flow, or that emits Gaussian tape, is an answer key for the wrong exam. The probe layer treats the simulator as a subject: realism certification against the stylized facts [Cont, 2001], payoff-boundary sweeps for manipulation, markout decomposition for adverse selection, and ecology runs that ask which strategies survive contact with each other.

One consequence is worth stating before the experiments. The first three principles are properties of committed code, checkable today by running the listed tests, and this paper claims them as such. The empirical section makes no analogous claim: its eight findings are defined, seeded and scripted, and their numbers arrive when the evidence run executes against the integrated kernel.

3 The environment

SharpeArena is a Rust workspace of three crates. The crate `sharpearena` holds everything determinism-critical: the stepping engine, the wire contract, the scenario generator, the limit-order-book matching engine, mandates and the execution-noise model, built on the published SharpeBench engine crates rather than a vendored copy. The crate `sharpearena-wasm` compiles the same kernel

to WebAssembly and tests it against the native engine. The crate `sharpearena-py` binds the engine into a Python package that adds the ecosystem adapters: Gymnasium [Towers et al., 2024], PettingZoo [Terry et al., 2021], Minari [Farama Foundation, 2024], a `verifiers` training environment, and an MCP server. The adapters are thin by policy; anything that must be byte-identical lives in Rust.

3.1 Point-in-time observation, and the guard behind it

The engine owns the time cursor. On each step it emits a `MarketObservation` whose per-symbol snapshot carries a trailing window of closes up to the decision date, with optional fundamentals and news channels derived from that same trailing window. There is no API on the data layer that returns a bar after the cursor, so a policy cannot observe a future bar: the property is the shape of the interface, not a rule. A property test iterates every scenario tier and asserts that no surfaced window extends past the cleared bar and that each window is a suffix of the true history.

One layer out, at the agent-harness boundary, sits `LookaheadGuard`, a refusal layer for tools and policies that try to read what the environment will not give. It carries a deny-list of named operations, each mapped to the reason it is refused: reading the raw dataset (the answer key), slicing past the current index, peeking the next bar, or feeding the scenario seed to the policy as a feature, which would identify the episode and trivialize generalization. Substring tells catch novel operation names, and an index check bounds any point-in-time slice: reading an index strictly greater than the current bar raises. A research escape hatch exists as an explicit environment variable, so relaxing the guard is a recorded decision rather than an accident. The guard is defense in depth behind the structural guarantee: a harness that bypasses it still cannot make the environment leak.

3.2 Determinism across runtimes

A scenario is a pure function of one 64-bit seed. The generator's price transform uses only multiply, add, divide and max; it never calls `ln` or `exp`, which differ across libm implementations, so a generated panel is byte-identical on any platform and any runtime. The commitment is pinned, not assumed: the source commits FNV-1a fingerprints of canonical generated scenarios, one per family at a fixed seed, and of a scripted order sequence's fill tape in the matching engine, whose tape carries only integers. Tests recompute the hashes on every commit. The WebAssembly crate closes the loop with two parity tests: the wasm kernel must reproduce the native engine's returns and trace for the same run, and its scenario generator must match the native output byte-for-byte and hash to the committed golden value. A published number is therefore the same number from Rust, the browser and Python.

3.3 Recompute-from-decisions tamper evidence

A rollout trace is a versioned JSONL artifact that records the agent's decisions and rejects anything leaky: the writer refuses values that would embed future data in the record. Everything else, returns included, is recomputed at verification time by replaying the decisions against the frozen scenario, and because the engine is deterministic the recompute is byte-identical. The npm package's test suite exercises the adversarial case directly: it captures a trajectory, doctors every order's target weight, replays both, and asserts the tampered artifact cannot reproduce the honest returns. An agent cannot lie about its performance to anyone willing to run the replay.

3.4 Procedural scenarios and disjoint seed bands

Scenario families follow Procgen's integer-seed-interval model [Cobbe et al., 2020]: Calm, Hard and Extreme volatility-and-jump tiers, a cointegrated-pairs family with a genuinely mean-reverting spread, and a regime-shift family that hands a trend regime over to a whipsaw. The Rust core exposes `train_test_split`, which carves a provably disjoint test family from a bounded train interval and refuses an unbounded one, and `cross_regime_split`, which holds the seed band fixed while swapping the regime, a strictly stronger robustness probe than a within-tier split. The

evaluation protocol fixes the canonical bands: train seeds $[0, 256)$, a gap of ten thousand seeds that is never sampled, and a held-out test band $[10256, 10512)$. The Python layer additionally reserves an absolute evaluation namespace starting at seed 10^6 and commits a named held-out regression set inside it, with a disjointness guard asserting the set never touches the train band. Overfitting is then a measurement: `generalization_gap` scores the same policy on train and held-out bands and reports the differenced deflated Sharpe, and `cross_regime_transfer` reports the in-distribution minus zero-shot out-of-distribution gap over identical seeds, which is zero by construction when the regimes coincide.

3.5 The task suite and the provable optimum

Beyond the canonical position-trading environment, the suite ships a simplex-allocation portfolio task, a VWAP/TWAP implementation-shortfall execution task, and two market models with genuine microstructure: an endogenous shared-book market where aggregate agent flow moves the cleared price through a Kyle permanent-impact term [Kyle, 1985] and an Almgren-Chriss temporary-impact term [Almgren and Chriss, 2001], and a deterministic limit-order-book market with integer ticks, price-time priority, a single-price call-auction uncross and a read-only walk-the-book sweep-cost query.

The market-making task is the suite's calibration instrument. It implements the Avellaneda-Stoikov model [Avellaneda and Stoikov, 2008] and ships the closed-form optimal quoting policy beside the environment: reservation price $r = s - q\gamma\sigma^2\tau$ and optimal half-spread $\delta^* = \gamma\sigma^2\tau/2 + \gamma^{-1}\ln(1 + \gamma/\kappa)$, skewed by inventory. Both the environment and the policy are pure functions of the same frozen parameter set, so the regret metric `mm_regret` runs the candidate and the optimum on identical seeds and reports the mean reward gap. An agent here is not scored against other agents; it is scored against a provable optimum, and the optimum scores approximately zero regret against itself by construction.

3.6 Mandates and deferred claims

Each scenario can sample a trading mandate, long-only, market-neutral, drawdown-capped or beat-a-benchmark, and the objective is conditioned on it: `mandate_breach` detects structural violations from the realized weights and returns, and the training rubric penalizes wrong-objective behavior rather than merely not rewarding it. A deterministic failure taxonomy classifies every episode into a single worst-case-first disposition, cascade-wiped before bankrupt before stopped-out before mandate breach before clean, and a suite-level rollup tallies the distribution with a schema that always carries every mode.

Questions that resolve after the episode get the same structural treatment as look-ahead. A `DeferredDesk` is the only object the agent touches: its entire state is a list of claims and one integer clock advanced by the harness, it holds no dataset and no environment reference, and it computes no score, so there is no code path from a claim to its resolving datum. Resolution is a free function over claims and outcomes that refuses any outcome available at or before the claim's commit bar and any outcome predating the stated horizon. The agent cannot see the answer at commit time because the object it holds cannot produce one.

3.7 The probe layer: distrusting the simulator

A synthetic market can flatter agents in ways real markets do not, so the suite probes itself.

Realism. A stylized-facts battery [Cont, 2001] computes excess kurtosis, absolute-return autocorrelation, Zumbach timescale asymmetry, gain/loss skew, aggregational Gaussianity and a Fano burstiness factor over a generated panel, and `certify_realism` grades the panel against directional believability bounds. A drifted generator that emits near-Gaussian tape fails the certification instead of silently letting agents win on unrealistic markets.

Manipulation. A crude push-and-unwind schedule is scored against its own zero-impact reference: the live and reference runs share the seed and the follower population, and differ only in that the reference sets both impact coefficients to zero, so the difference in the manipulator's profit is exactly what moving the price was worth. A boundary sweep varies one impact parameter and interpolates where the pump stops paying, and a size-response sweep asks whether payoff is bounded in push size. A specification whose manipulation profit rises monotonically at the largest size tested is flagged as broken, because real books get more expensive to push than the push is worth.

Adverse selection. Informed meta-order flow is routed against a maker roster, with fills struck at the pre-move mid so the maker's markout carries the informed trader's edge. The design is a paired control: the informed leg sides child orders on the alpha, the uninformed leg sides the identical flow on a coin while holding the price path fixed, and the comparison reports mean markout per filled unit at each horizon. If the informed leg does not mark out worse, the module's own numbers are declared noise.

Ecology. A replicator dynamic runs a population of strategies over generations: fitness earned in a shared-book market where a species that gains share becomes a larger fraction of the flow everyone else fills against, seat allocation from population shares, extinction below a threshold, an innovation operator that breeds parameter-perturbed variants, and shock schedules that rotate regimes or scale impact to simulate liquidity droughts. The control schedule holds the environment steady, so any claim about a shock has a baseline to beat.

4 The agent contract and its governance

An agent is an external program in any language that reads a `MarketObservation` as JSON and replies with a `Decision` as JSON. The harness sends an observation, the agent returns a decision, repeat. Everything else in this paper exists to make that loop trustworthy; this section is the loop's promise of stability.

4.1 Authoritative artifacts

The contract has four authoritative artifacts, all committed. `CONTRACT_VERSION`, currently `1.0`, is the single source of truth for the wire version and lives in the contract module of the core crate. Two JSON Schemas, draft 2020-12, define `MarketObservation` and `Decision` for non-Rust implementers. A conformance kit of fixture documents, covering the normal multi-symbol case, a single symbol, an empty-portfolio start, signed-weight shorting and a legacy decision shape, is exercised by a committed conformance test. And a governance document states the evolution rules in writing.

4.2 Additive-only evolution

The contract evolves additively. The only backwards-compatible change is a new field that is optional with a default, so an agent written against an older version keeps parsing newer observations and the harness keeps parsing older decisions. The optional fields are absent from the schemas' `required` lists by construction. Removing, renaming or retyping a field, making an optional field required, and removing or renaming an action variant are all defined as breaking and forbidden on the v1 surface. A breaking change never mutates the v1 types in place; it ships a parallel namespace with its own schemas and its own major version, and the two run side by side through a published deprecation window. Adding a new action variant is itself classified as breaking, because consumers that exhaustively match would misparse it, so it also goes through the parallel namespace rather than an additive bump.

4.3 Version semantics

`CONTRACT_VERSION` versions the wire shape only and is deliberately decoupled from the crate and package versions, which move on their own release cadence. A major bump is a breaking change

Table 1: Baseline leaderboard under the corrected kernel: deflated Sharpe (DSR) and pass^k rate per tier, 16 seeds each. No policy is rank-eligible on any tier: eligibility requires pass^k on every seed, and the best rate is 0.75.

	Calm		Hard		Extreme	
Policy	DSR	pass^k	DSR	pass^k	DSR	pass^k
kelly_vol_target	1.0000	0.75	0.0277	0.31	0.0029	0.12
equal_weight_long	1.0000	0.62	0.1114	0.31	0.0234	0.06
min_variance	0.9849	0.44	0.0024	0.19	0.0243	0.06
flat	0.0007	0.00	0.0007	0.00	0.0007	0.00
momentum	0.0000	0.00	0.0000	0.00	0.0000	0.06
max_sharpe	0.0000	0.06	0.0000	0.00	0.0000	0.00

shipped as a parallel namespace. A minor bump is reserved for a batch of additive fields significant enough to advertise; a single additive field needs no bump at all, because existing agents are unaffected by definition. A package release never, on its own, bumps the contract version.

4.4 Why the contract can be trusted without trusting the host

The governance promise is social; the properties of Sections 3.1 to 3.3 are technical, and together they remove the host from the trust equation. A leaderboard entry produced on this environment can be recomputed by anyone from the frozen seed and the submitted decisions, on any platform, to the byte. The host's role reduces to publishing artifacts, and the artifacts carry their own verification. The scoring rules themselves are not defined here at all: the rank key is the SharpeBench kernel's deflated Sharpe with its reliability and process gates [Toca, 2026, Bailey and de Prado, 2014], applied to the recomputed return series, so the environment and the scorer can be audited independently and neither can quietly redefine the other.

5 Experiments

Eight findings. Each subsection states the experiment's definition (the entry point, the seeds, the quantity measured), then reports the numbers from the committed evidence run. Every experiment is a committed script under `paper/src/` that writes its result as JSON to `paper/evidence/` and its figure as a PDF under `paper/figures/`; the exact commands are listed in Section A. Scenario seeds are fixed in the scripts, so the evidence run is deterministic given the pinned kernel (SharpeBench 0.5.0 under `sharpearena` 0.8.0, the version recorded in the evidence).

5.1 F1: The corrected deflation prior rewrites the baseline board

The reference-policy field (flat, equal-weight long, momentum, minimum-variance, maximum-Sharpe, fractional-Kelly volatility targeting) is rolled over sixteen scenario seeds in each of the Calm, Hard and Extreme tiers via `run_baselines`. Each policy's pooled return series is scored by the SharpeBench `score_run` kernel with the declared trial count equal to the field size, and each row carries a seed-paired bootstrap 95% confidence interval on the deflated Sharpe (2,000 resamples).

Under the pre-0.5.0 kernel this board was all zeros: every baseline scored a deflated Sharpe of 0.0000 on every tier. That was not strictness. The kernel applied its annualized 0.5 deflation prior per period without conversion, which set the bar near an annualized Sharpe of 18 on daily bars, a level nothing clears. This is the same unit bug the companion SharpeBench paper reports as its Finding 1 [Toca, 2026]; the 0.5.0 kernel states the prior annualized and converts it once at 252 periods per year.

Table 1 and Figure 1 show the corrected board. On Calm, drift is free deflated Sharpe: `kelly_vol_target` and `equal_weight_long` saturate the DSR at 1.0000 (bootstrap CIs $[0.9798, 1.0000]$ and $[0.8343, 1.0000]$) and `min_variance` reaches 0.9849, though with a near-vacuous interval of $[0.0264, 1.0000]$. Yet nothing on the board is rank-eligible, because no policy

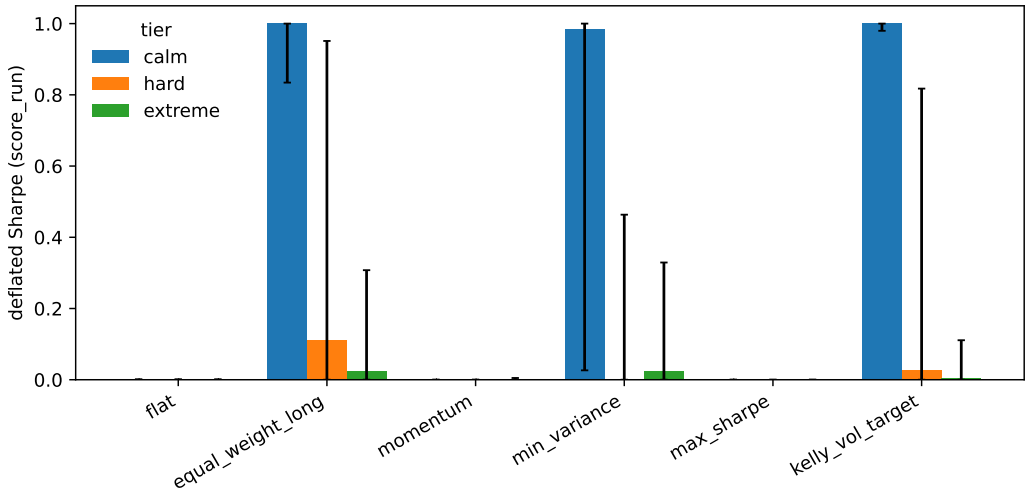


Figure 1: Deflated Sharpe per baseline per tier with seed-paired bootstrap CIs. Calm saturates the long baselines; Hard and Extreme collapse them toward zero.

passes the per-run gate on every seed; the best pass^k rate is 0.75. On Hard the best DSR collapses to 0.1114 and on Extreme to 0.0243, and the long baseline's pass^k rate degrades monotonically with stress (0.62, 0.31, 0.06). `momentum` and `max_sharpe` lose money outright on every tier (mean per-bar returns down to -1.6×10^{-3} on Extreme).

The reading: eligibility is a conjunction, and each leg catches what the other misses. Deflation alone would crown drift on Calm; pass^k alone would miss the overfit breadth the deflation discounts. The earlier all-zeros board looked strict but was measuring a unit error. The corrected board is the honest zero: the number a real agent has to beat is a positive, deflated Sharpe that survives every held-out seed, and no reference policy produces one.

5.2 F2: Regret against the closed-form optimum

On the Avellaneda-Stoikov market-making task, `mm_regret` runs each candidate quoting policy and the analytically optimal policy on the same sixteen seeded episodes ($\sigma = 2.0$, $\gamma = 0.1$, $\kappa = 1.5$, 200 steps) and reports the mean reward gap to the optimum. Candidates are the optimum itself and fixed-spread quoters over a grid of half-spreads.

The optimum's regret is 0.0 by construction, which validates the metric's zero point. The fixed-spread curve is U-shaped (Figure 2): regret is 84.6 at a half-spread of 0.05 (quoting too tight, filled constantly, run over by inventory), falls to a minimum of 0.037 at 0.5, and rises again to 36.6 at 2.0 and 58.8 at 4.0 (quoting too wide, earning nothing). The best fixed spread comes within 0.04 of the optimum on this parameterization, but only at the one grid point that happens to match the closed-form spread; every other choice pays double-digit regret for ignoring inventory and time-to-go.

The point of shipping the optimum is that agents on this task are scored on regret, not raw PnL. A market-making leaderboard ranked on PnL rewards whichever policy drew the friendliest flow; regret against a provable optimum on the same episodes removes that luck axis entirely.

5.3 F3: Generalization gap and cross-regime transfer

`generalization_gap` scores the default reference policy (equal-weight long) on disjoint train and held-out seed bands (16 seeds each, separated by a 10,000-seed gap) within each tier. `cross_regime_transfer` then holds sixteen seeds fixed and varies the regime, producing the

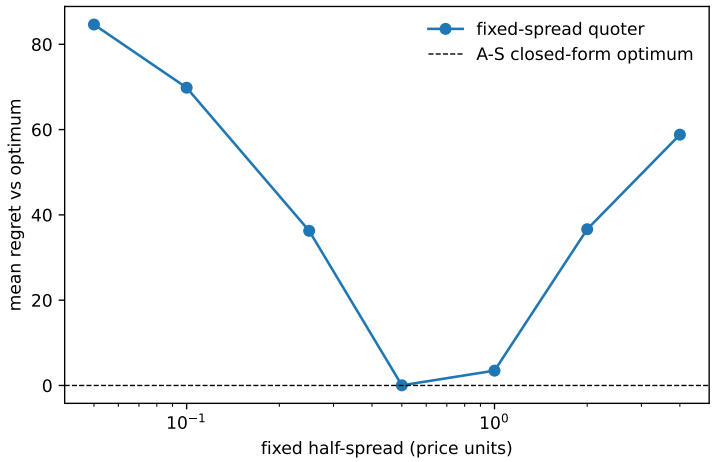


Figure 2: Mean regret versus the closed-form Avellaneda-Stoikov optimum over 16 paired episodes. Fixed-spread quoting is U-shaped in the half-spread; the optimum defines zero.

Table 2: Left: within-tier generalization gap (train minus test deflated Sharpe) for the equal-weight reference. Right: cross-regime transfer gap (in-distribution minus zero-shot out-of-distribution DSR), rows train, columns test.

Tier	Train DSR	Test DSR	Gap
Calm	1.0000	0.9988	$+0.0012$
Hard	0.1231	0.4618	-0.3388
Extreme	0.0269	0.1280	-0.1012

	Calm	Hard	Extreme
Calm	0.000	$+0.877$	$+0.973$
Hard	-0.877	0.000	$+0.096$
Extreme	-0.973	-0.096	0.000

full three-by-three transfer matrix whose diagonal is zero by construction, because identical regimes reuse byte-identical environments.

Table 2 reports both. The within-tier gaps are the expected control: the reference policy has no fitted parameters, so it cannot overfit its train band, and the instrument correctly reads near zero on Calm ($+0.0012$) and noise-signed values on the stressed tiers (the negative Hard and Extreme gaps mean the held-out band happened to be kinder; with no learning, the gap is pure seed noise, and its magnitude, up to 0.34 on Hard, calibrates how much band luck a 16-seed evaluation carries).

The transfer matrix (Figure 3) is where regime structure appears. Training regime Calm and testing zero-shot on Hard costs 0.877 of deflated Sharpe; Calm to Extreme costs 0.973, essentially the whole score. The matrix is antisymmetric because the policy is fixed, and its off-diagonal magnitude is the quantity a within-tier gap is structurally blind to: a policy scored only on calm seeds can carry a saturated DSR into a leaderboard while being worth 0.03 under stress. Both metrics are cheap, and the protocol requires reporting both.

5.4 F4: Stylized-facts realism certification

For each tier, price panels from eight seeded episodes are graded by `certify_realism` against the directional bounds: positive excess kurtosis, positive absolute-return autocorrelation, positive aggregational Gaussianity, and a super-Poisson Fano factor, with gain/loss skew and Zumbach asymmetry reported as informational [Cont, 2001].

Table 3 and Figure 4 report the certification honestly, and it is a mixed verdict. The fat-tail facts split cleanly by tier, as they should: Calm's returns are platykurtic (mean excess kurtosis -0.97, aggregational Gaussianity -0.58; a low-volatility, jump-free regime is expected to sit near Gaussian, and the gate correctly refuses to certify it as fat-tailed tape), while Hard and Extreme pass excess kurtosis and aggregational Gaussianity on all eight seeds (means $+16.3$ and $+11.4$ excess kurtosis).

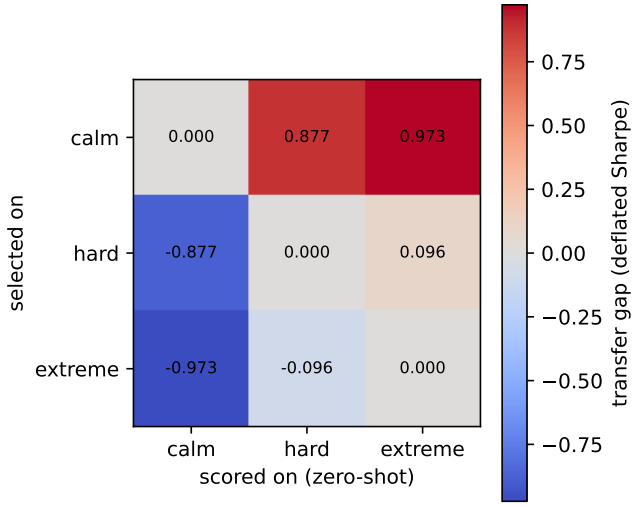


Figure 3: Cross-regime transfer matrix for the reference policy: out-of-distribution deflated Sharpe with the seed band held fixed.

Table 3: Stylized-facts certification per tier: mean fact value over 8 seeds, per-fact pass counts, and the all-facts pass rate. Calm is near-Gaussian by design and fails the fat-tail facts; the stressed tiers pass them.

	Excess kurtosis		$\lvert r\rvert$ autocorr		Fano factor		All facts
Tier	mean	pass	mean	pass	mean	pass	pass rate
Calm	-0.97	0/8	$+0.0002$	5/8	1.19	7/8	0.000
Hard	$+16.30$	8/8	-0.0107	1/8	0.85	0/8	0.000
Extreme	$+11.38$	8/8	-0.0118	1/8	0.79	1/8	0.125

But the full conjunction almost never passes: 0/8 on Calm, 0/8 on Hard, 1/8 on Extreme. The binding facts on the stressed tiers are volatility clustering (absolute-return autocorrelation is slightly negative at this 121-bar episode length) and the Fano factor, which drops sub-Poisson exactly when jumps concentrate variance in few bars. This is a finding against the generator, stated as such: the vol-and-jump construction buys fat tails without buying persistent volatility, and a future generator revision must add clustering (for example a stochastic-volatility driver) before the certification gate passes at tier level.

That driver now exists as an opt-in `vol_clustering` knob on the scenario spec: when positive, a deterministic EMA persistence state driven by realized absolute returns modulates each bar's return deviation, using the same mul/add/div/max arithmetic as the tier transforms so cross-runtime byte-identity is preserved, and a committed golden hash pins the clustered output alongside the unclustered tiers. At `vol_clustering` $= 0$ (the default and the canonical leaderboard configuration) the generated panels are byte-identical to the tapes certified above, so the finding stands as reported. Re-certifying the same tiers and seeds at strength 0.5 flips the volatility-clustering fact: the absolute-return autocorrelation pass count rises from 5/8 to 8/8 on Calm, 1/8 to 8/8 on Hard, and 1/8 to 5/8 on Extreme (Hard's mean $\lvert r\rvert$ autocorrelation moves from -0.011 to $+0.029$), lifting Hard's all-facts pass rate from 0.000 to 0.250. The remaining binding fact on the stressed tiers is the sub-Poisson Fano factor, which the clustering driver improves (Hard 0.85 to 0.90) but does not yet fix.

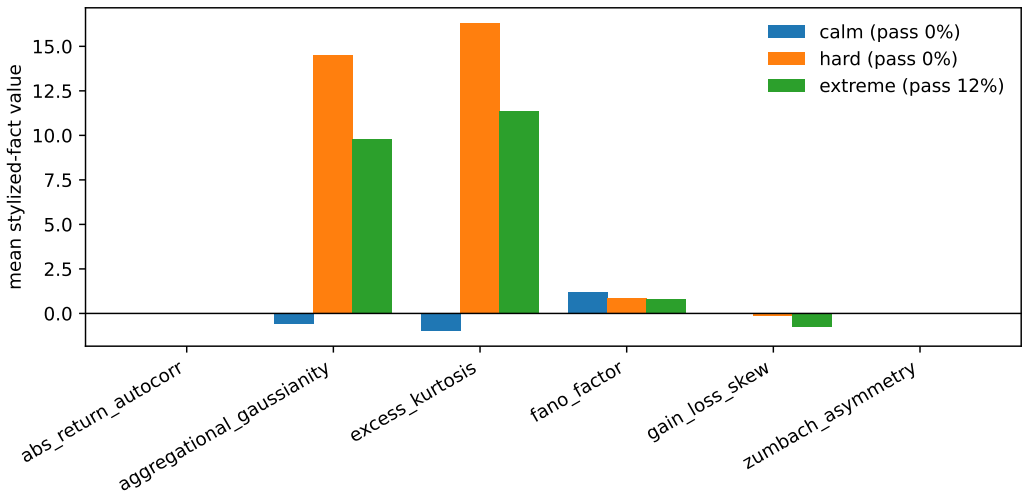


Figure 4: Per-seed stylized-fact values by tier against the certification bounds.

Table 4: Manipulation probe. Left: impact PnL (in units of 10^{-4}, per-bar) at the endpoints of each swept axis; every axis is unprofitable everywhere, so no profitability boundary exists to report. Right: size response in the push weight.

Axis	Low end	High end	Boundary
Kyle λ (0 to 0.8)	-1.3	-31.7	none
η (0 to 0.8)	-2.2	-22.4	none
Follower gain (0 to 120)	-2.6	-6.7	none

Push weight	Impact PnL (10^{-4})
0.10	-0.14
0.25	-0.50
0.50	-1.53
1.00	-5.14

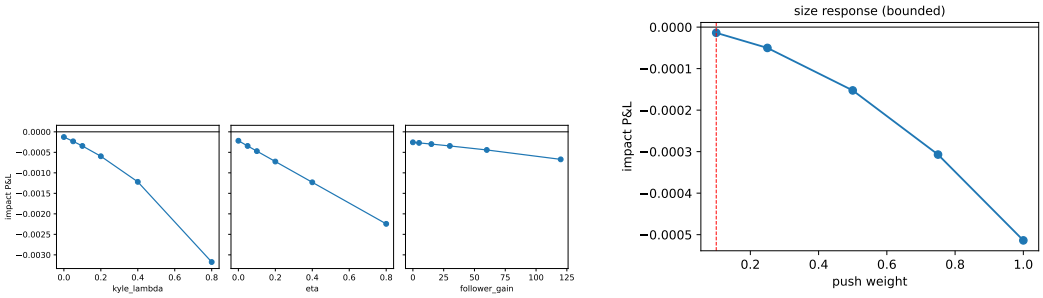
5.5 F5: The manipulation boundary and size response

A push-and-unwind manipulator is scored against its zero-impact paired reference (identical seed and schedule, $\lambda = \eta = 0$) over eight seeds, so `impact_pnl` attributes profit specifically to having moved the price. `impact_boundary_sweep` sweeps permanent impact (Kyle λ), temporary impact (η) and the momentum-follower gain; `size_response` sweeps the push weight.

The result (Table 4, Figure 5) is the red-team outcome the impact specification should produce. On every axis the pump loses money at every sampled coefficient, including the realistic center of the grid ($\lambda = 0.1$, $\eta = 0.05$: impact PnL -3.4×10^{-4}), and the loss deepens monotonically as impact or follower gain rises: more impact means a worse average entry on the push and a worse exit on the unwind, and the follower population never front-loads enough momentum flow to pay for it. The size response is bounded and decreasing: the least-bad size is the smallest sampled push weight (0.1, -0.14×10^{-4}) and the loss grows roughly linearly to -5.1×10^{-4} at full size. There is no interior optimum where manipulation pays and scales. Had the sweep found a profitable region at plausible coefficients, that would be a defect finding against the environment; instead the probe certifies that the Kyle-plus-Almgren-Chriss book prices this manipulation as a cost at every tested point. The module ships with an explicit disclaimer: it is a diagnostic on the simulator, not a strategy.

5.6 F6: Adverse-selection markout decomposition

`compare_informed_vs_uninformed` runs twenty-four paired episodes in which the identical meta-order flow is sided on the alpha in one leg and on a coin flip in the other, holding schedule, sizes and price path fixed, and reports mean maker markout per filled unit at horizons of 1, 5 and 20 bars.



(a) Boundary sweeps per impact axis.

(b) Size response in the push weight.

Figure 5: The manipulation probe: impact-attributed PnL is negative everywhere and worsens with impact strength and with size.

Table 5: Maker markout per filled unit against informed versus paired-uninformed flow, 24 paired episodes. The gap is the price of trading against information, and it grows with horizon.

Horizon (bars)	Informed	Uninformed	Gap
1	0.689	0.761	0.072
5	0.489	0.791	0.302
20	0.386	0.823	0.436

The machinery is non-vacuous in the required direction at every horizon (Table 5, Figure 6): makers filled by informed flow mark out worse than the paired-uninformed control at $h = 1$, 5 and 20, and the per-unit gap widens from 0.072 to 0.436 as the information realizes, which is the signature shape of adverse selection (the loss is in the drift after the fill, not in the fill itself).

The single-episode maker decomposition sharpens this. The wider quoter (`maker_0`, 135 fills) keeps a positive markout at $h = 20$ (0.526 per unit, toxic-fill rate 0.296): the flow is not selecting against it. The tighter quoter (`maker_1`, 92 fills) shows the subsidised pattern: aggregate markout is still positive at $h = 20$ (0.197 per unit) but its meta-order fills alone lose 0.197 per unit on 62% of its filled quantity, with a toxic-fill rate of 0.533 at $h = 20$, so noise flow is paying for the informed flow. Spread capture and adverse drift decompose exactly (`spread_capture` + `adverse_drift` = `markout`): `maker_1` captures 82.8 of spread and gives back 64.7 to adverse drift by $h = 20$.

5.7 F7: Failure-mode distributions

Every rollout of the eight-policy field (the six reference policies plus the two behavioral counterparties), across all three tiers and sixteen seeds (384 episodes), is classified by `classify_episode_failure` into the worst-case-first taxonomy: cascade-wiped, bankrupt, stopped-out, mandate breach (structural, drawdown or inventory), or clean, with a per-scenario sampled mandate and a 50% drawdown stop in force.

Table 6 and Figure 7 show the shape of failure as stress rises. The clean rate falls from 0.56 on Calm to 0.39 on Extreme. Structural mandate breaches are roughly flat across tiers (36, 36, 31), which is correct: a shorting policy under a sampled long-only mandate breaks the rule regardless of the weather. The stress-sensitive modes are the drawdown family: mandate drawdown breaches triple from 11 on Calm to 32 on Extreme, and hard stop-outs at the 50% line appear only on Extreme (5 episodes, four of them `max_sharpe`). No policy in this field ever reaches bankruptcy or a cascade wipe; the taxonomy's worst states are reachable but not by these baselines at this stop level. Per policy, `flat` is clean 48/48 by construction, while `overconfident` is the worst citizen on Calm (7 of its 16 episodes end in a drawdown breach) and `max_sharpe` is the worst on Extreme (3 clean of 16). The headline for entrants: on this field, half of everything that goes wrong is process, not PnL, which is exactly the half a return-ranked board never sees.

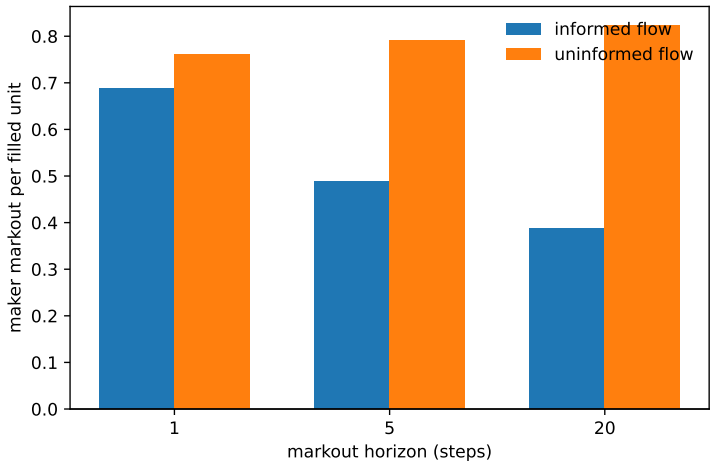


Figure 6: Markout per filled unit by horizon: informed flow versus the paired-uninformed control.

Table 6: Failure-mode counts per tier, 128 episodes each (8 policies × 16 seeds). No episode reaches bankruptcy or a cascade wipe; failure is dominated by mandate breaches, and drawdown breaches triple from Calm to Extreme.

Tier	Clean	Stopped out	Mandate struct.	Mandate DD	Mandate inv.	Bankrupt	Clean rate
Calm	72	0	36	11	9	0	0.56
Hard	68	0	36	14	10	0	0.53
Extreme	50	5	31	32	10	0	0.39

5.8 F8: Ecology outcomes under shocks

`run_ecology` seeds the replicator with the eight baseline species over a shared-book market via `market_payoffs` (12 generations, field size 8, innovation every 4 generations breeding parameter-perturbed variants of the leader) and runs matched trajectories under a steady all-Calm control schedule and a `regime_shocks` schedule rotating Calm (generations 0 to 3), Hard (4 to 7) and Extreme (8 to 11).

The two schedules select different winners from the same founding field (Table 7, Figure 8). Under the control, `kelly_vol_target` sweeps the board: it peaks at a 1.00 share by generation 7 and ends dominant at 0.81, with its own bred variants (`target_vol8`, `kelly_fraction12`) holding the persistent remainder and all seven other founders extinct. Under the shock schedule the same species collapses: it peaks at 0.649 in generation 3, the last Calm generation, and is gone by generation 5 once the regime turns Hard. Dominance passes to `equal_weight_long` (final share 0.67, peak 0.75 in generation 8), with its three bred variants persistent (combined 0.33) and the behavioral counterparties eliminated early on both schedules (`overconfident` is extinct by generation 1 in both). `detect_coalitions` flags 8 coalitions in the control and 7 under shocks; the strongest are parent-variant pairs at correlation 1.00, the bred clones moving with their parent, with the tightest cross-species pair at 0.997.

The direction of the divergence is explained by the strategy parameters: volatility targeting is the sharpest exploiter of a stationary calm regime and precisely the leverage profile that a Hard transition liquidates, while the unlevered long book survives the rotation. That is the finding the ecology probe exists to produce: which strategies survive transitions is a different question from which scores best inside a regime, and one generation of regime change reorders the answer.

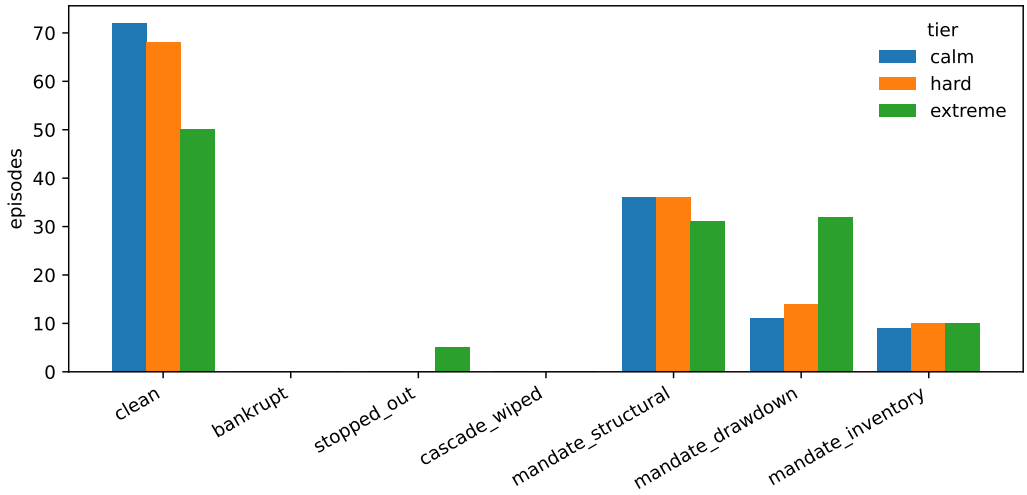


Figure 7: Failure-mode distribution per tier per policy over 384 classified episodes.

Table 7: Classified ecology outcomes, control versus shocked (11 species each: 8 founders plus 3 bred variants).

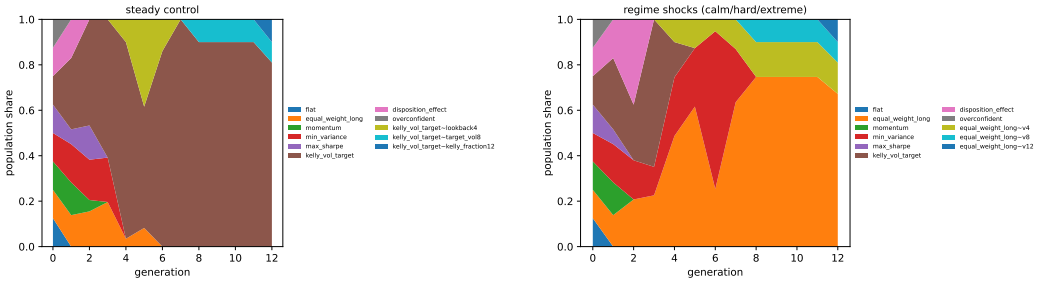
Schedule	Dom.	Pers.	Coll.	Ext.	Dominant species (final share)
Control (all Calm)	1	2	1	7	kelly_vol_target (0.81)
Shocked (Calm/Hard/Extreme)	1	3	3	4	equal_weight_long (0.67)

6 Related work

Environment interfaces. Gym [Brockman et al., 2016] demonstrated that a small, stable environment interface is worth more than any single environment, and Gymnasium [Towers et al., 2024] carried the interface forward with versioned environment IDs and a conformance checker; SharpeArena's Python surface is a first-class Gymnasium citizen with versioned, namespaced IDs and adopts its `check_env`. PettingZoo [Terry et al., 2021] standardized the multi-agent case, which SharpeArena's shared-book and batched-competition environments implement. Minari [Farama Foundation, 2024] standardizes offline-RL datasets; SharpeArena exports leak-safe rollouts to it. The wire contract of Section 4 differs from all of these in scope: it is a language-agnostic JSON boundary for external agent processes, governed like a protocol rather than a library API.

Evaluation rigor in RL. The environment's evaluation stance imports three lessons from the RL evaluation literature. From the ALE revisitation [Machado et al., 2018]: determinism in the environment invites trajectory memorization, so evaluation must be designed against it; SharpeArena keeps determinism for verifiability and defeats memorization with procedural scenario distributions and held-out seed bands instead of injected stochasticity. From Procgen [Cobbe et al., 2020]: procedural generation over integer seed intervals makes generalization measurable, which Section 3.4 ports to markets. Prioritized Level Replay [Jiang et al., 2021] shows seed-level curricula shape what agents learn, which motivates keeping the train band operator-owned and the held-out band frozen. τ-bench [Yao et al., 2024] argues that agent reliability must be measured across repeated runs rather than on average; that requirement lives in the SharpeBench kernel this environment feeds [Toca, 2026].

Market simulation. ABIDES [Byrd et al., 2020] is the closest simulator in spirit: a high-fidelity multi-agent market with a matching engine and background agent populations. SharpeArena trades some of that fidelity for properties ABIDES does not target: cross-runtime byte-determinism with committed golden hashes, a structurally leak-free observation boundary, replay-based tamper ev-



(a) Control: steady Calm.

(b) Shocked: Calm, then Hard, then Extreme.

Figure 8: Population-share trajectories over 12 replicator generations. The regime rotation replaces the calm-regime winner with the unlevered long book.

idence, and a governed external-agent wire contract. The microstructure models are the classical ones: Kyle's permanent impact [Kyle, 1985], Almgren-Chriss temporary impact and execution cost [Almgren and Chriss, 2001], and the Avellaneda-Stoikov quoting model with its closed-form optimum [Avellaneda and Stoikov, 2008]. The realism gate certifies the generator against Cont's stylized facts [Cont, 2001].

Scoring. The environment deliberately contains no scorer. The deflated Sharpe [Bailey and de Prado, 2014] and the reliability and process gates that consume this environment's trajectories are specified in the SharpeBench paper [Toca, 2026]; this paper is the producer half of that funnel.

7 Limitations

One synthetic generator family. All three tiers come from a single procedural vol-and-jump generator; no real price series enters the environment, and the tiers differ in that generator's parameters, not in kind. F4 quantifies the cost: the tape has fat tails on the stressed tiers but no volatility clustering at episode length, and the full stylized-facts conjunction passes on only 1 of 24 certified runs. An agent that excels here has demonstrated skill against this generator's structure, not against a market, and the realism gate says so rather than hiding it.

The baselines are trivial policies, not trained agents. Every number in Section 5 is produced by fixed reference policies (buy-and-hold analogs, one-step tilts, closed-form allocators) and two deliberately biased behavioral counterparties. They establish the honest zero and calibrate the instruments; they do not establish what a trained agent scores. In particular, the near-zero within-tier generalization gaps in F3 are the expected control for policies with nothing to overfit, not evidence that the environment resists overfitting by learners.

The Python probe layer is outside the golden-hash guarantee. The byte-identical claim covers the Rust core and its WebAssembly and Python bindings, where golden hashes and parity tests enforce it. The probe layer that produced F4 through F8 (realism, manipulation, adverse selection, failure classification, ecology) is deterministic in its seeds through NumPy's seeded generators, but it is off the golden-hash path, and float reproducibility there is the platform's, not the paper's.

The impact models are stylized. The endogenous market composes linear Kyle and Almgren-Chriss terms; the adverse-selection module deliberately does not separate the meta-order's information from its impact; the manipulation probe's follower population is a one-parameter caricature of momentum chasing. F5's clean verdict, manipulation unprofitable everywhere, is a statement about this specification at the tested coefficients, and a stylized model can fail realistically for the wrong reason.

The guard is a deny-list. The structural guarantee covers the environment's own surface. `LookaheadGuard` extends it to harness tools by pattern, and a deny-list is exactly as good as its patterns; a novel leak path through a side channel the environment does not own, wall-clock time, filesystem state, a network call, is out of scope for both layers. The replay check verifies decisions against frozen data and therefore catches falsified results, not information an agent obtained elsewhere.

No live external entrants yet. The contract, the conformance kit and the replay verifier exist, and the evidence run exercises them, but no third party has yet submitted an agent. A hosted intake, a live leaderboard and multi-tenant isolation of untrusted agent processes do not exist, and the governance document is a written promise backed by tests that has not been stressed by a hostile or careless external implementer.

8 Reproducibility statement

The environment is a Rust workspace of three crates under the MIT and Apache-2.0 licenses, published to crates.io, npm and PyPI, with a pinned toolchain and a lockfile, depending on the published SharpeBench engine crates rather than a vendored copy. The core forbids unsafe code. The determinism claims are enforced rather than asserted: committed FNV-1a golden hashes pin the generated scenarios and the matching engine's fill tape, the WebAssembly crate's tests assert byte-identity with the native engine for both stepping and scenario generation, and the npm package's smoke test performs the replay-and-tamper check of Section 3.3 on every run. The contract artifacts, schemas, conformance fixtures and the governance document, are committed beside the code they govern, and a conformance test exercises the fixtures. Continuous integration covers the Rust surface with formatting, lints as errors, tests and a WASM target build, plus dependency auditing, the npm package and the Python wheel.

Every number in Section 5 is produced by a committed script under `paper/src/` from fixed seeds, writing JSON evidence to `paper/evidence/` and figures to `paper/figures/`; the commands are listed in Section A. The scripts import the published Python package and call only its public API, so reproducing the evidence requires the package version pinned in Section A and nothing else.

No large language model is a component of the environment, the contract or any reported result. The environment has no judge. Language models were used to draft and edit prose; every bibliography entry was checked against its source.

References

Robert Almgren and Neil Chriss. Optimal execution of portfolio transactions. *Journal of Risk*, 3(2): 5–39, 2001.

Marco Avellaneda and Sasha Stoikov. High-frequency trading in a limit order book. *Quantitative Finance*, 8(3):217–224, 2008.

David H. Bailey and Marcos López de Prado. The deflated sharpe ratio: Correcting for selection bias, backtest overfitting, and non-normality. *The Journal of Portfolio Management*, 40(5):94–107, 2014.

Greg Brockman, Vicki Cheung, Ludwig Pettersson, Jonas Schneider, John Schulman, Jie Tang, and Wojciech Zaremba. OpenAI Gym, 2016.

David Byrd, Maria Hybinette, and Tucker Hybinette Balch. ABIDES: Towards high-fidelity multi-agent market simulation, 2020.

Karl Cobbe, Christopher Hesse, Jacob Hilton, and John Schulman. Leveraging procedural generation to benchmark reinforcement learning. In *Proceedings of the 37th International Conference on Machine Learning*, 2020.

Rama Cont. Empirical properties of asset returns: Stylized facts and statistical issues. *Quantitative Finance*, 1(2):223–236, 2001.

Farama Foundation. Minari: A dataset API for offline reinforcement learning. `https://minari.farama.org`, 2024.

Minqi Jiang, Edward Grefenstette, and Tim Rocktäschel. Prioritized level replay. In *Proceedings of the 38th International Conference on Machine Learning*, 2021.

Albert S. Kyle. Continuous auctions and insider trading. *Econometrica*, 53(6):1315–1335, 1985.

Marlos C. Machado, Marc G. Bellemare, Erik Talvitie, Joel Veness, Matthew Hausknecht, and Michael Bowling. Revisiting the arcade learning environment: Evaluation protocols and open problems for general agents. *Journal of Artificial Intelligence Research*, 61:523–562, 2018.

Jordan Terry, Benjamin Black, Nathaniel Grammel, Mario Jayakumar, Ananth Hari, Ryan Sullivan, Luis S. Santos, Clemens Dieffendahl, Caroline Horsch, Rodrigo Perez-Vicente, Niall Williams, Yashas Lokesh, and Praveen Ravi. Pettingzoo: Gym for multi-agent reinforcement learning. In *Advances in Neural Information Processing Systems*, volume 34, 2021.

Tiberiu Toca. SharpeBench: The Luck-Robust Benchmark for AI Trading Agents, 2026. Preprint.

Mark Towers, Ariel Kwiatkowski, Jordan Terry, John U. Balis, Gianluca De Cola, Tristan Deleu, Manuel Goulão, Andreas Kallinteris, Markus Krimmel, Arjun KG, Rodrigo Perez-Vicente, Andrea Pierré, Sander Schulhoff, Jun Jet Tai, Hannah Tan, and Omar G. Younis. Gymnasium: A standard interface for reinforcement learning environments, 2024.

Shunyu Yao, Noah Shinn, Pedram Razavi, and Karthik Narasimhan. τ-bench: A benchmark for tool-agent-user interaction in real-world domains, 2024.

A Commands that produce every number

All commands run from the repository root at the tagged version, with the `sharpearena` Python package installed. The package version is recorded in every evidence file (`sharpearena` 0.8.0 over the SharpeBench 0.5.0 kernel for this run). Each evidence script fixes its seeds internally, writes JSON to `paper/evidence/` and, where a figure carries the finding, a PDF to `paper/figures/`.

The design-property checks (Sections 3.1 to 3.3 and 4).

```
cargo test -p sharpearena              # golden hashes, leak-free
                                       # property tests, conformance kit
cargo test -p sharpearena-wasm         # native == wasm parity + golden
npm test --prefix npm/sharpearena      # replay + tamper detection
python -m pytest crates/sharpearena-py # Python surface, check_env,
                                       # seed-band disjointness
```

F1: baseline leaderboard (Section 5.1).

```
python paper/src/make-f1-baselines.py
  -> paper/evidence/f1-baselines.json, paper/figures/f1-baselines.pdf
```

F2: regret vs the closed-form optimum (Section 5.2).

```
python paper/src/make-f2-regret.py
  -> paper/evidence/f2-regret.json, paper/figures/f2-regret.pdf
```

F3: generalization gap and cross-regime transfer (Section 5.3).

```
python paper/src/make-f3-generalization.py
  -> paper/evidence/f3-generalization.json,
     paper/figures/f3-transfer-matrix.pdf
```

F4: stylized-facts realism certification (Section 5.4).

```
python paper/src/make-f4-realism.py
  -> paper/evidence/f4-realism.json, paper/figures/f4-realism.pdf
```

F5: manipulation boundary and size response (Section 5.5).

```
python paper/src/make-f5-manipulation.py
  -> paper/evidence/f5-manipulation.json,
     paper/figures/f5-boundaries.pdf, paper/figures/f5-size-response.pdf
```

F6: adverse-selection markouts (Section 5.6).

```
python paper/src/make-f6-adverse-selection.py
  -> paper/evidence/f6-adverse-selection.json,
     paper/figures/f6-markouts.pdf
```

F7: failure-mode distributions (Section 5.7).

```
python paper/src/make-f7-failures.py
  -> paper/evidence/f7-failures.json, paper/figures/f7-failures.pdf
```

F8: ecology under shocks (Section 5.8).

```
python paper/src/make-f8-ecology.py
  -> paper/evidence/f8-ecology.json,
     paper/figures/f8-ecology-control.pdf,
     paper/figures/f8-ecology-shocked.pdf
```

Regenerating every figure from the committed evidence.

```
python paper/src/make-figures.py
```

Each `make-f*` script produces its own figure at run time; `make-figures.py` re-renders all figures from the JSON already in `paper/evidence/` without re-running any experiment, so a reader can check that no figure contains a number the evidence does not.